

**САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ
ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО**

Дисциплина: Бэк-энд разработка

Отчет

**Лабораторная работа №1 «Реализация REST API на основе
boilerplate»**

Выполнила:

Клапнева Диана

Группа К3342

Проверил:

Добряков Д. И.

Санкт-Петербург

2026 г.

Задача

Задание:

1. Используйте любой boilerplate, который считаете наиболее подходящим для ваших нужд (можно взять и мой)
2. С учётом имеющихся данных по результатам ДЗ1 и ДЗ2 реализуйте:
 - модели
 - представления
 - контроллеры
3. В результате выполнения лабораторной работы вы должны продемонстрировать полностью работающее API согласно вашего варианта

Необходимо сделать отчёт по шаблону

Ход работы

Шаг 1: Начнем с моделей – структуры данных, по которой будут строиться и таблицы в бд. Пропишем их файлы в папке /models. Entity это сущность. Все будем проверять на примере авторизации, поэтому посмотрим на сущность пользователя.

> Этот компьютер > Документы > GitHub > backend-lab1 > src > models			
	Имя	Дата изменения	Тип
1	Deal.entity	27.07.2026 14:08	Файл "TS"
↑	Estate.entity	27.07.2026 14:05	Файл "TS"
↑	Message.entity	27.07.2026 14:09	Файл "TS"
↑	Session.entity	27.07.2026 14:08	Файл "TS"
↑	User.entity	27.07.2026 14:04	Файл "TS"
↑	user.subscriber	27.07.2026 13:17	Файл "TS"

В начале файла – декораторы из библиотеки typeORM, потом импорт других таблиц т.к. они связаны, далее идут поля самой сущности.

```

User.entity.ts
C:\Users\local\Documents\GitHub\backend-lab1\tsconfig.json {} "user.entity"

1  import {
2      Entity,
3      PrimaryGeneratedColumn,
4      Column,
5      CreateDateColumn,
6      UpdateDateColumn,
7      OneToMany
8  } from 'typeorm';
9  import { Estate } from './Estate.entity';
10 import { Deal } from './Deal.entity';
11 import { Session } from './Session.entity';
12
13 export enum DealRole {
14     LANDLORD = 'landlord',
15     TENANT = 'tenant'
16 }
17
18 @Entity('users')
19 export class User {
20     @PrimaryGeneratedColumn()
21     id: number;
22
23     @Column({ name: 'first_name' })
24     first_name: string;
25
26     @Column({ name: 'middle_name', nullable: true })
27     middle_name?: string;
28
29     @Column({ name: 'last_name' })
30     last_name: string;
31
32     @Column({ type: 'enum', enum: DealRole, nullable: true })
33     deal_role?: DealRole;
34
35     @Column({ unique: true })
36     email: string;
37
38     @Column()
39     password: string;

```

Шаг 2: DTO (data transfer object) нужен для валидации формата данных.

```

user.dto.ts
C:\Users\local\Documents\GitHub\backend-lab1\tsconfig.json {} "user.dto"

1  import { IsString, IsOptional, IsEnum, IsBoolean, MinLength, MaxLength } from 'class-validator';
2  import { DealRole } from '../models/User.entity';
3
4 references
export class CreateUserDto {
    @IsString()
    @MaxLength(50)
    first_name: string;

    @IsOptional()
    @IsString()
    middle_name?: string;

    @IsString()
    @MaxLength(50)
    last_name: string;

    @IsOptional()
    @IsEnum(DealRole)
    deal_role?: DealRole;

    @IsEmail()
    email: string;

    @IsString()
    @MinLength(8)
    password: string;
}

4 references
export class UpdateUserDto {
    @IsOptional()
    @IsString()
    first_name?: string;

    @IsOptional()
    @IsString()
    middle_name?: string;

    @IsOptional()
    @IsString()
    last_name?: string;

    @IsOptional()
    @IsEnum(DealRole)
    deal_role?: DealRole;

    @IsOptional()
    @IsEmail()
    email?: string;

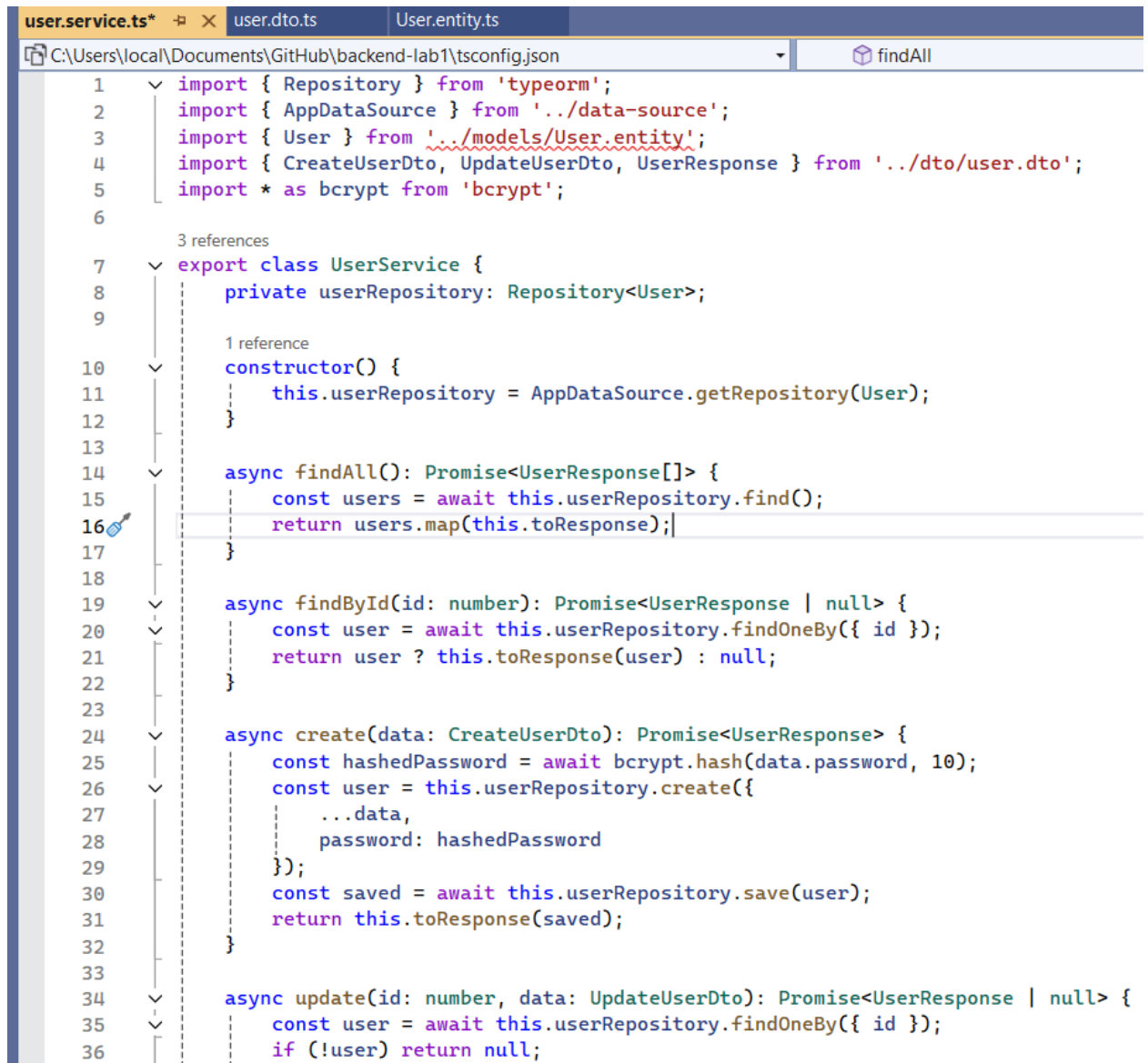
    @IsOptional()
    @IsString()
    password?: string;

    @IsOptional()
    @IsBoolean()
    is_verified?: boolean;
}

7 references
export class UserResponse {
    id: number;
    first_name: string;
    middle_name?: string;
    last_name: string;
    deal_role?: DealRole;
    email: string;
    is_verified: boolean;
    created_at: Date;
    updated_at?: Date;
}

```

Шаг 3: Уже сейчас для разных сущностей прописаны свои сервисы. В файле `user.service.ts` прописаны все методы, которые вызывают контроллеры. Например, в `create` мы не только создаем запись, но и хешируем пароль пользователя.



```
1  import { Repository } from 'typeorm';
2  import { AppDataSource } from '../data-source';
3  import { User } from '../models/User.entity';
4  import { CreateUserDto, UpdateUserDto, UserResponse } from '../dto/user.dto';
5  import * as bcrypt from 'bcrypt';
6
7  export class UserService {
8      private userRepository: Repository<User>;
9
10     constructor() {
11         this.userRepository = AppDataSource.getRepository(User);
12     }
13
14     async findAll(): Promise<UserResponse[]> {
15         const users = await this.userRepository.find();
16         return users.map(this.toResponse);
17     }
18
19     async findById(id: number): Promise<UserResponse | null> {
20         const user = await this.userRepository.findOneBy({ id });
21         return user ? this.toResponse(user) : null;
22     }
23
24     async create(data: CreateUserDto): Promise<UserResponse> {
25         const hashedPassword = await bcrypt.hash(data.password, 10);
26         const user = this.userRepository.create({
27             ...data,
28             password: hashedPassword
29         });
30         const saved = await this.userRepository.save(user);
31         return this.toResponse(saved);
32     }
33
34     async update(id: number, data: UpdateUserDto): Promise<UserResponse | null> {
35         const user = await this.userRepository.findOneBy({ id });
36         if (!user) return null;
```

Шаг 4: Прописываем контроллеры – они обрабатывают http запросы клиента, вызывают сервисы и отправляют response. Тот же пример с пользователем – `user.controller.ts`, в нем также прописан JSDoc комментарий для того, чтобы метод был виден в документации `swagger ui`. Возможные ошибки для пользователя – 404(не нашли его), 422(одна из проблем валидации данных), 500(проблема с сервером), а 200 – успех.

```

/**
 * @swagger
 * /users:
 *   post:
 *     summary: Create a new user
 *     tags: [Users]
 *     requestBody:
 *       required: true
 *       content:
 *         application/json:
 *           schema:
 *             $ref: '#/components/schemas/CreateUserRequest'
 *     responses:
 *       201:
 *         description: User created
 *         content:
 *           application/json:
 *             schema:
 *               $ref: '#/components/schemas/User'
 *       422:
 *         description: Validation error
 *         content:
 *           application/json:
 *             schema:
 *               $ref: '#/components/schemas/Error'
 */

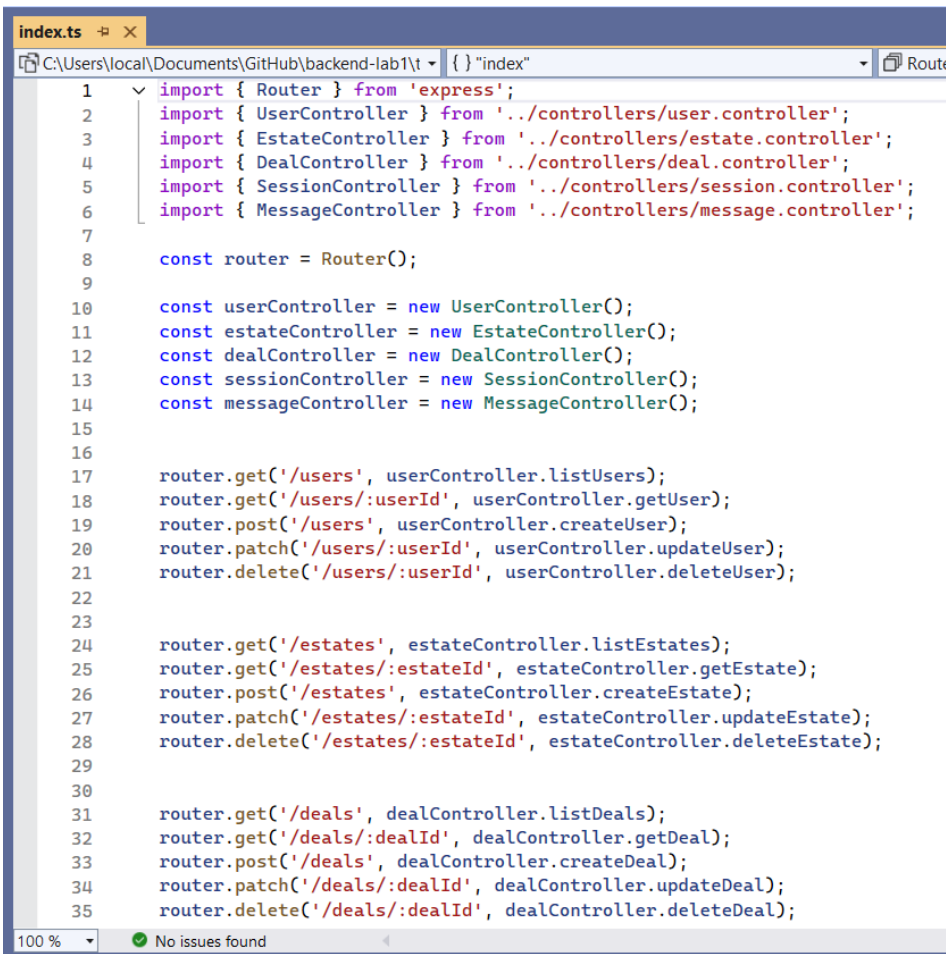
createUser = async (req: Request, res: Response): Promise<void> => {
  try {
    const dto = plainToInstance(CreateUserDto, req.body);
    const errors = await validate(dto);

    if (errors.length > 0) {
      res.status(422).json({
        statusCode: 422,
        error: {
          code: 'VALIDATION_ERROR',
          message: 'Validation failed',
          details: errors.map(e => ({
            field: e.property,
            message: Object.values(e.constraints || {})[0]
          })))
        }
      });
      return;
    }

    const user = await this.userService.create(dto);
    res.status(201).json(user);
  } catch (error: any) {
    if (error.code === '23505') {
      res.status(422).json({
        statusCode: 422,
        error: {
          code: 'VALIDATION_ERROR',
          message: 'Email already exists'
        }
      });
      return;
    }
    res.status(500).json({ error: 'Internal server error' });
  }
};

```

Шаг 5: В папке /routes прописаны все маршруты в index.ts.

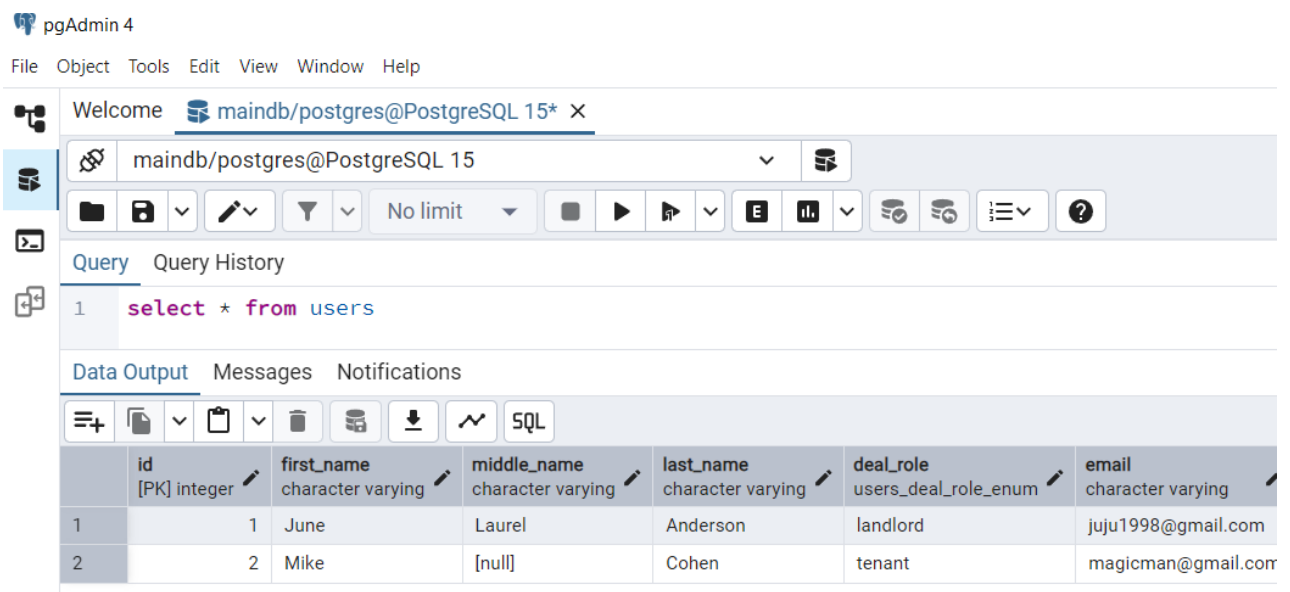
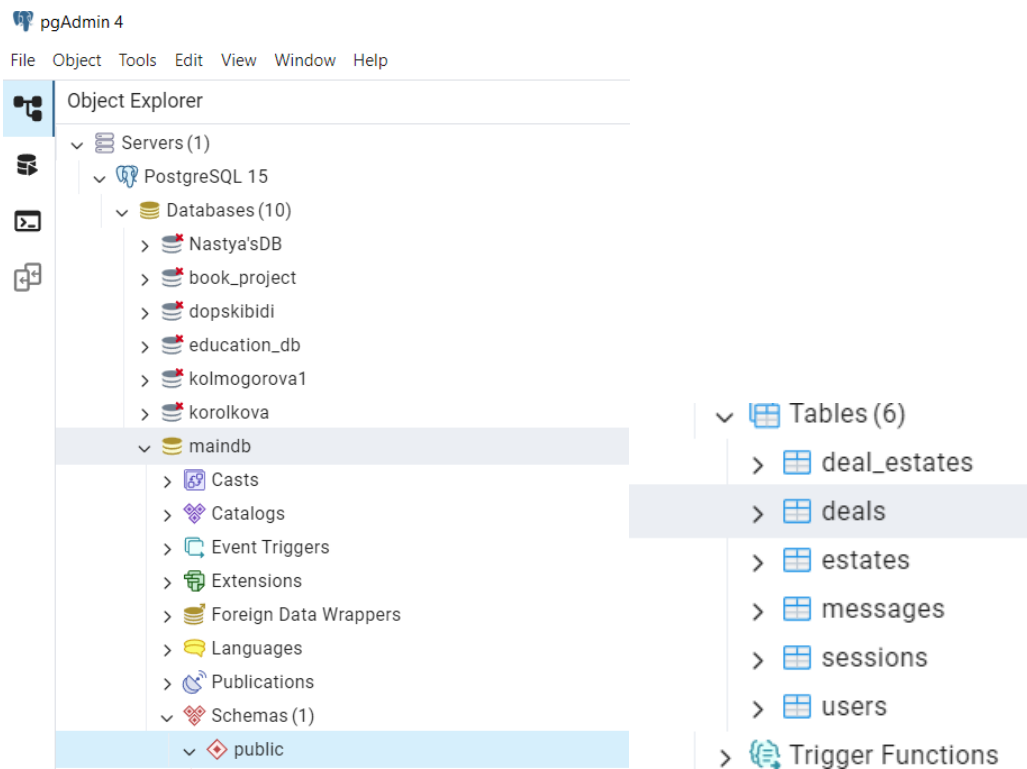


```

index.ts
C:\Users\local\Documents\GitHub\backend-lab1\t { } "index"
1  import { Router } from 'express';
2  import { UserController } from '../controllers/user.controller';
3  import { EstateController } from '../controllers/estate.controller';
4  import { DealController } from '../controllers/deal.controller';
5  import { SessionController } from '../controllers/session.controller';
6  import { MessageController } from '../controllers/message.controller';
7
8  const router = Router();
9
10 const userController = new UserController();
11 const estateController = new EstateController();
12 const dealController = new DealController();
13 const sessionController = new SessionController();
14 const messageController = new MessageController();
15
16
17 router.get('/users', userController.listUsers);
18 router.get('/users/:userId', userController.getUser);
19 router.post('/users', userController.createUser);
20 router.patch('/users/:userId', userController.updateUser);
21 router.delete('/users/:userId', userController.deleteUser);
22
23
24 router.get('/estates', estateController.listEstates);
25 router.get('/estates/:estateId', estateController.getEstate);
26 router.post('/estates', estateController.createEstate);
27 router.patch('/estates/:estateId', estateController.updateEstate);
28 router.delete('/estates/:estateId', estateController.deleteEstate);
29
30
31 router.get('/deals', dealController.listDeals);
32 router.get('/deals/:dealId', dealController.getDeal);
33 router.post('/deals', dealController.createDeal);
34 router.patch('/deals/:dealId', dealController.updateDeal);
35 router.delete('/deals/:dealId', dealController.deleteDeal);
100 %  No issues found

```

Шаг 6: С помощью pgAdmin4 создаем базу данных maindb, подключение к ней осуществляем с помощью файла data-source.ts.



```
data-source.ts  user.controller.ts  user.service.ts*  user.dto.ts  User.entity.ts
C:\Users\local\Documents\GitHub\backend-lab1\tsconfig.json  { } "data-source"

1  import { DataSource } from 'typeorm';
2  import { User } from '../models/User.entity';
3  import { Estate } from '../models/Estate.entity';
4  import { Deal } from '../models/Deal.entity';
5  import { Session } from '../models/Session.entity';
6  import { Message } from '../models/Message.entity';
7
8  export const AppDataSource = new DataSource({
9      type: 'postgres',
10     host: 'localhost',
11     port: 5432,
12     username: 'postgres',
13     password: 'postgres',
14     database: 'maindb',
15     synchronize: true,
16     logging: false,
17     entities: [User, Estate, Deal, Session, Message],
18     migrations: [],
19     subscribers: [],
20 });
```

Общий алгоритм реализации запросов такой:

- 1) Клиент делает http запрос
- 2) В src\routes\index.ts определяется по маршруту какой контроллер вызывается
- 3) В src\controllers контроллер определяет нужный сервер и какой именно метод он должен выполнить
- 4) Сервис в src\services выполняет действие обращаясь к модели данных
- 5) В src\models как раз эти модели , они описывают таблицы, к которым подключаемся
- 6) В файле data-source.ts прописано подключение к бд через postgres
- 7) Pgadmin4 делает запрос к бд и возвращает ответ в формате json.

Шаг 7: Запускаем сервис с помощью командной строки. Прописанные в index.ts строки выводятся в окне – значит, мы можем посмотреть на документацию api в Swagger UI.

C:\WINDOWS\system32\cmd.exe

Your environment has been set up for using Node.js 24.18.0 (x64) and npm.

C:\Users\local>cd C:\Users\local\Documents\GitHub\backend-lab1

C:\Users\local\Documents\GitHub\backend-lab1>npm run dev

> backend-lab1@1.0.0 dev

> nodemon

[nodemon] 2.0.22

[nodemon] to restart at any time, enter `rs`

[nodemon] watching path(s): src***

[nodemon] watching extensions: ts

[nodemon] starting `ts-node src/index.ts`

Data Source иницирован

Сервер: http://localhost:3000

Swagger UI: http://localhost:3000/api-docs

Статус работы: http://localhost:3000/health

Переходим по адресу <http://localhost:3000/api-docs/> и видим нашу документацию.

Real Estate API - Lab 1 1.0.0 OAS 3.0

Документация API для управления недвижимостью

[Contact API Support](#)

Servers

http://localhost:3000/api/v1 - Development server

Users

- GET** /users Get all users
- POST** /users Create a new user
- GET** /users/{userId} Get user by ID
- PATCH** /users/{userId} Update user by ID
- DELETE** /users/{userId} Delete user by ID

Sessions

Schemas

- User >
- CreateUserRequest >
- UpdateUserRequest >
- Estate >
- CreateEstateRequest >
- UpdateEstateRequest >
- Deal >

Модели данных тоже описаны здесь.

Я создала записи пользователей – арендодателя и арендатора, объект недвижимости и сделку об аренде между ними через соответствующие методы POST. Теперь через GET мы можем на все это посмотреть, а через PATCH – редактировать.

Responses

Curl

```
curl -X 'GET' \
'http://localhost:3000/api/v1/users' \
-H 'accept: application/json'
```

Request URL

```
http://localhost:3000/api/v1/users
```

Server response

Code	Details
------	---------

200	
-----	--

Response body

```
[
  {
    "id": 1,
    "first_name": "June",
    "middle_name": "Laurel",
    "last_name": "Anderson",
    "deal_role": "landlord",
    "email": "jujul998@gmail.com",
    "is_verified": false,
    "created_at": "2026-07-28T12:28:37.704Z",
    "updated_at": "2026-07-28T12:28:37.704Z"
  },
  {
    "id": 2,
    "first_name": "Mike",
    "middle_name": null,
    "last_name": "Cohen",
    "deal_role": "tenant",
    "email": "magicman@gmail.com",
    "is_verified": false,
    "created_at": "2026-07-28T12:29:56.575Z",
    "updated_at": "2026-07-28T12:29:56.575Z"
  }
]
```

Curl

```
curl -X 'GET' \
'http://localhost:3000/api/v1/estates/1' \
-H 'accept: application/json'
```

Request URL

```
http://localhost:3000/api/v1/estates/1
```

Server response

Code	Details
------	---------

200	
-----	--

Response body

```
{
  "id": 1,
  "user_id": 1,
  "address": "middle of nowhere",
  "type": "cottage",
  "room_amount": 6,
  "description": "old ass pre-revolution era at the edge of town =",
  "price": 0,
  "image_path": "no",
  "bath_type": null,
  "fridge": true,
  "washing_machine": false,
  "internet": true,
  "tv": false,
  "furnished_rooms": true,
  "furnished_kitchen": true,
  "is_verified": false,
  "is_available": true,
  "created_at": "2026-07-28T12:33:10.258Z",
  "updated_at": "2026-07-28T12:33:10.258Z"
}
```

Вывод

Данная лабораторная работа позволяет спланировать структуру api проекта и понять на практике, как в принципе реализуются запросы клиента к серверу. Благодаря этой работе я более детально представляю себе будущую архитектуру приложения.